

Convolutional Neural Networks (CNN)

1. Why CNN for Images?

MLP flattens images into 1D vectors, destroying spatial relationships between pixels. A pixel at (0,0) and (27,27) are treated equally. CNN preserves 2D structure, processing images as grids where neighbor relationships matter.

1.1 MLP vs CNN Parameter Efficiency

```
MLP: nn.Linear(784, 32) = 784×32 + 32 = 25,120 parameters
```

```
CNN: nn.Conv2d(1, 32, 3) = 1×3×3×32 + 32 = 320 parameters
```

CNN uses 78× fewer parameters for similar capacity!

This efficiency comes from weight sharing — the same small filter is reused across the entire image.

2. Convolution Operation — The Math

2.1 How Convolution Works

A filter (kernel) slides across the image. At each position, it computes an element-wise multiplication with the image patch, then sums all values to produce one output number.

Image patch (3×3):		Filter (3×3):
[1, 0, 1]		[1, 0, 1]
[0, 1, 0]	×	[0, 1, 0]
[1, 0, 1]		[1, 0, 1]

Output = $1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 = 4$

2.2 Conv2d Parameters Explained

```
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1, stride=1)
```

in_channels=3: RGB image has 3 channels

out_channels=32: 32 different filters, each produces one feature map

kernel_size=3: each filter is 3×3

padding=1: adds 1-pixel border of zeros (keeps size same)

stride=1: filter moves 1 pixel at a time

2.3 3D Convolution for RGB Images

For RGB input, each filter is actually 3D — one 3×3 slice per channel:

RGB Image:	One filter (3×3×3):
R [28×28]	R slice [3×3]
G [28×28]	G slice [3×3] → produces ONE feature map
B [28×28]	B slice [3×3]

Computation: slide each slice across its channel,
sum all three partial results + bias = one output value

2.4 Parameter Count

```
nn.Conv2d(3, 32, kernel_size=3):
```

Each filter: $3 \times 3 \times 3 = 27$ weights + 1 bias = 28
32 filters: $32 \times 28 = 896$ total parameters

3. What Filters Learn

Filters are NOT designed by us — they are LEARNED during training through backpropagation.

3.1 Hierarchical Feature Learning

Layer 1: edges, corners, gradients	(3×3 receptive field)
Layer 2: textures, simple shapes patterns)	(combines Layer 1
Layer 3: object parts (eyes, wheels, handles) patterns)	(combines Layer 2
Layer 4: whole objects (faces, cars, animals) patterns)	(combines Layer 3

3.2 Example Filter Patterns

Horizontal edge detector:	Vertical edge detector:
[-1, -1, -1]	[-1, 0, 1]
[0, 0, 0]	[-1, 0, 1]
[1, 1, 1]	[-1, 0, 1]

High output where horizontal edge exists, zero on flat area	High output where vertical edge exists, zero on flat area
--	--

4. Padding

4.1 Why Padding?

Without padding, each convolution shrinks the image. A 3×3 filter on a 32×32 image produces 30×30 output. After several layers, the image becomes too small.

4.2 Output Size Formula

```
output_size = (input + 2*padding - kernel) / stride + 1
```

```
Without padding: (32 + 0 - 3) / 1 + 1 = 30 (shrinks!)
```

```
With padding=1: (32 + 2 - 3) / 1 + 1 = 32 (same size!)
```

```
Rule of thumb: padding = kernel_size // 2 keeps size same
```

5. Stride

Stride controls how many pixels the filter jumps at each step.

```
Stride=1: filter moves 1 pixel at a time (default)
```

```
Input 32×32 → Output 32×32 (with padding=1)
```

```
Stride=2: filter jumps 2 pixels at a time
```

```
Input 32×32 → Output 16×16 (halves size, like MaxPool!)
```

```
(32 + 2 - 3) / 2 + 1 = 16
```

6. MaxPool2d — Downsampling

6.1 How It Works

```
MaxPool2d(2): takes maximum in each 2×2 window
```

```
[4, 3, 2, 1]      [6, 2]
[5, 6, 1, 0]  →   [2, 8]
[2, 1, 8, 3]
[0, 2, 1, 7]
```

6.2 Why MaxPool Not AveragePool?

MaxPool keeps the strongest activation — "was this feature detected?" AvgPool averages all values, diluting strong signals.

```
Edge detected at one spot: [0, 0, 9, 0]
```

```
MaxPool → 9 (edge preserved!)
```

```
AvgPool → 2.25 (edge diluted)
```

6.3 What MaxPool Provides

Reduces spatial size → less computation. Provides translation invariance → feature detected anywhere in window counts. Keeps strongest signals → most important features survive.

7. Receptive Field

The receptive field is the area of the original image that each neuron "sees." It grows with each layer.

After Conv Layer 1 (3×3):	each neuron sees 3×3 of input
After MaxPool:	each neuron sees 6×6 of input
After Conv Layer 2 (3×3):	each neuron sees 10×10 of input
After MaxPool:	each neuron sees 20×20 of input

Deeper layers see MORE of the image → detect LARGER patterns

This is why deeper layers can detect whole objects while shallow layers only detect edges.

8. Why Filters Increase (32 → 64 → 128)

8.1 Pattern Complexity

Simple patterns (edges) need few filters. Complex patterns (object parts) need many different filters to capture variety.

8.2 Computational Balance

Block 1:	32 filters × 32×32 = 32,768 values
Block 2:	64 filters × 16×16 = 16,384 values
Block 3:	128 filters × 8×8 = 8,192 values

Total values DECREASE even though filters increase!

As image shrinks through pooling, we can afford more filters without increasing total computation.

9. Weight Sharing

The same filter slides across the ENTIRE image. A horizontal edge filter detects horizontal edges everywhere — top-left, center, bottom-right. This is fundamentally different from MLP where each position has its own weights.

MLP: pixel (0,0) has weight w_1 , pixel (0,1) has weight w_2 , ...
784 different weights for 784 pixels

CNN: same 3×3 filter (9 weights) reused at ALL positions
A cat in top-left uses same filter as cat in bottom-right

Benefits: far fewer parameters, and features learned in one location transfer to all locations automatically.

10. BatchNorm2d and Dropout2d

10.1 BatchNorm2d vs BatchNorm1d

Aspect	BatchNorm1d	BatchNorm2d
Input shape	(batch, features)	(batch, channels, H, W)
Normalizes per	Feature	Channel
Used after	nn.Linear	nn.Conv2d

10.2 Dropout vs Dropout2d

Aspect	Dropout	Dropout2d
Drops	Individual values	Entire feature maps
Why	Each value independent	Nearby pixels correlated
Used in	Linear layers	Conv layers

11. Data Augmentation

Randomly modifies training images each epoch to create variety and prevent overfitting. Applied ONLY to training data, never test.

Transform	What It Does	What Model Learns
RandomHorizontalFlip	Mirrors image left-right	Left/right doesn't matter
RandomCrop	Crops random region	Handles position changes
ColorJitter	Changes brightness/contrast	Handles lighting changes
Normalize	Scales to standard range	Consistent input statistics

Forces model to learn actual FEATURES (ears, wheels) instead of memorizing exact pixel values.

12. Complete CNN Architecture Pattern

```
Input (3, 32, 32)
↓
[Conv(3→32) → BN2d → ReLU → Conv(32→32) → BN2d → ReLU → MaxPool →
Dropout2d]
↓ (32, 16, 16)
[Conv(32→64) → BN2d → ReLU → Conv(64→64) → BN2d → ReLU → MaxPool →
Dropout2d]
↓ (64, 8, 8)
[Conv(64→128) → BN2d → ReLU → Conv(128→128) → BN2d → ReLU → MaxPool →
Dropout2d]
↓ (128, 4, 4)
Flatten → (2048,)
```

```

↓
[Linear(2048, 256) → BN1d → ReLU → Dropout]
↓
[Linear(256, 10)] → no activation (CrossEntropyLoss handles softmax)

```

13. Transfer Learning with CNN

Pretrained CNNs (ResNet, EfficientNet) have already learned universal features from millions of images. We reuse the conv layers (feature extraction) and replace only the classification head for our task.

```

Feature Extraction: freeze ALL conv layers, train only new head
Fine-tuning:       freeze early layers, train later layers + head

```

Why it works:

```

Early layers (edges, textures) are universal → reusable
Later layers (object parts) are task-specific → need adaptation

```

Different learning rates:

```

Pretrained layers: lr = 0.0001 (gentle, preserve knowledge)
New head:         lr = 0.001  (aggressive, learn fast)

```

14. Architecture Comparison

Aspect	MLP	CNN	LSTM
Best for	Tabular data	Images	Text/sequences
Key layer	nn.Linear	nn.Conv2d	nn.LSTM
Processes	Flat vectors	2D spatial grids	Sequences
Parameters	Many (all connected)	Few (weight sharing)	Medium (recurrent)
Understands	Feature relationships	Spatial patterns	Word order/context

15. Summary

Convolution: sliding filter computes dot product with image patches to create feature maps. Each filter detects one type of pattern.

Weight sharing: same filter reused across entire image. A cat detected in any position uses the same learned filter. Makes CNN 28× more parameter-efficient than MLP.

Receptive field: deeper layers see larger areas of the input. Layer 1 sees 3×3 (edges), layer 3 sees 20×20 (whole objects).

MaxPool: keeps strongest activation in each window, providing downsampling and translation invariance.

Filters increase (32→64→128) because deeper patterns are more complex, while image size decreases to maintain computational balance.

Data augmentation only on training: creates variety to prevent memorization while keeping test evaluation fair and consistent.